AI VIET NAM
@aivietnam.edu.vn

Kiến trúc Apache Kafka

Từ broker đến partition, replication và offset

AI VIET NAM – AIO2026

Data&Code

AI VIET NAM
@aivietnam.edu.vn

Mục tiêu buổi học

❖Sau buổi học, bạn có thể

1. **Đọc** một sơ đồ Kafka: producer, broker, controller, consumer
2. **Phân biệt** partition với replica, Replicas với ISR
3. **Giải thích** key ảnh hưởng thế nào đến partition và thứ tự
4. **Theo dõi** tiến độ đọc bằng offset, committed offset và lag
5. **Lập luận** vì sao consumer group có giới hạn mở rộng

Tài liệu đi kèm
- Bài đọc nền tảng
- Lab 90 phút
- Quiz 10 câu

Mốc tài liệu: Apache Kafka 4.1

Outline

SECTION 1

Kafka và kiến trúc

SECTION 2

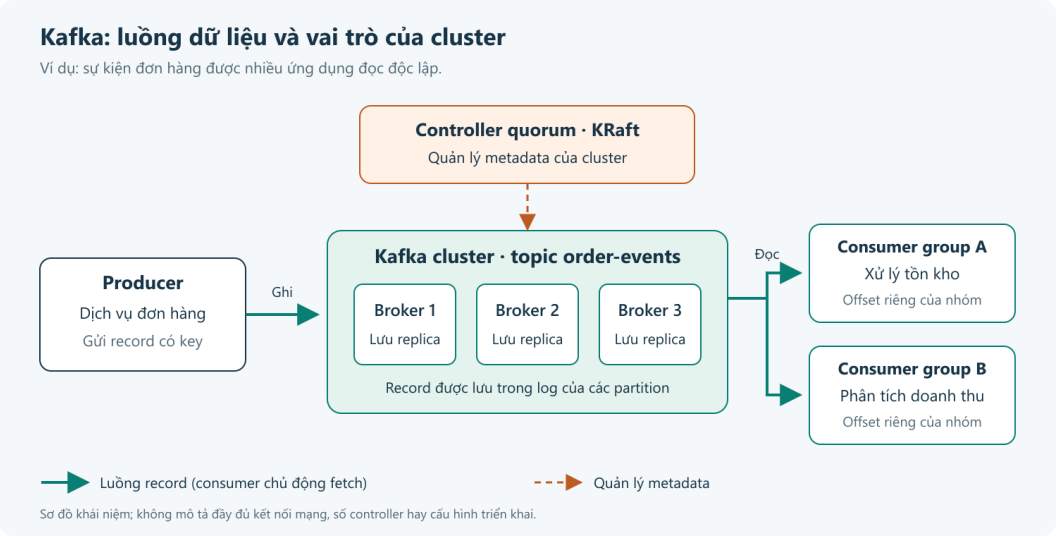
Topic, partition và key

SECTION 3

Replication và ISR

SECTION 4

Offset và consumer group

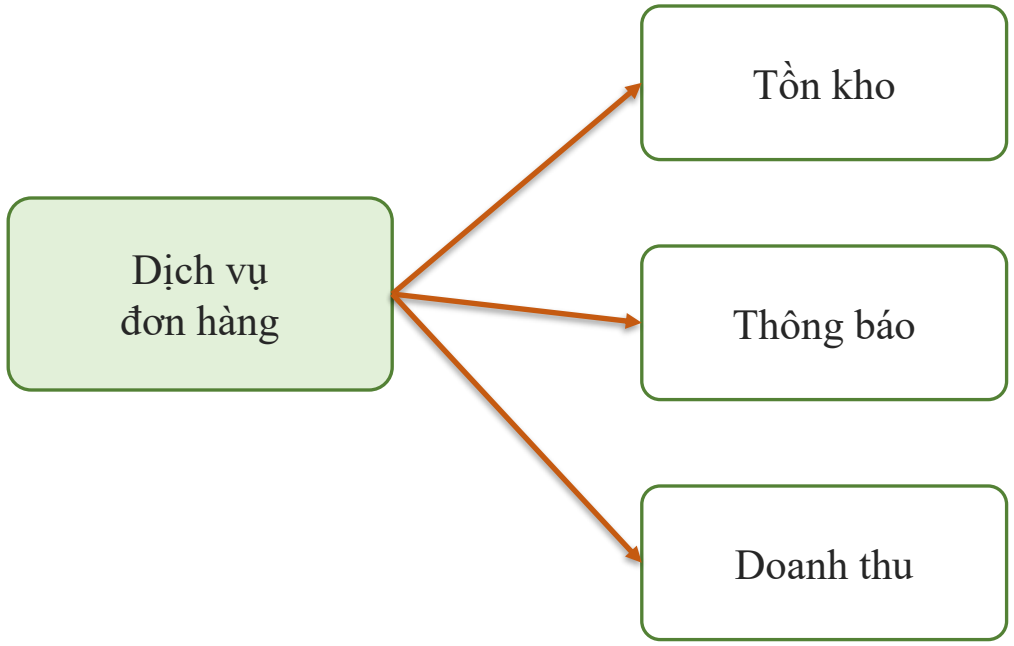


AI VIET NAM
@aivietnam.edu.vn

Kafka và kiến trúc

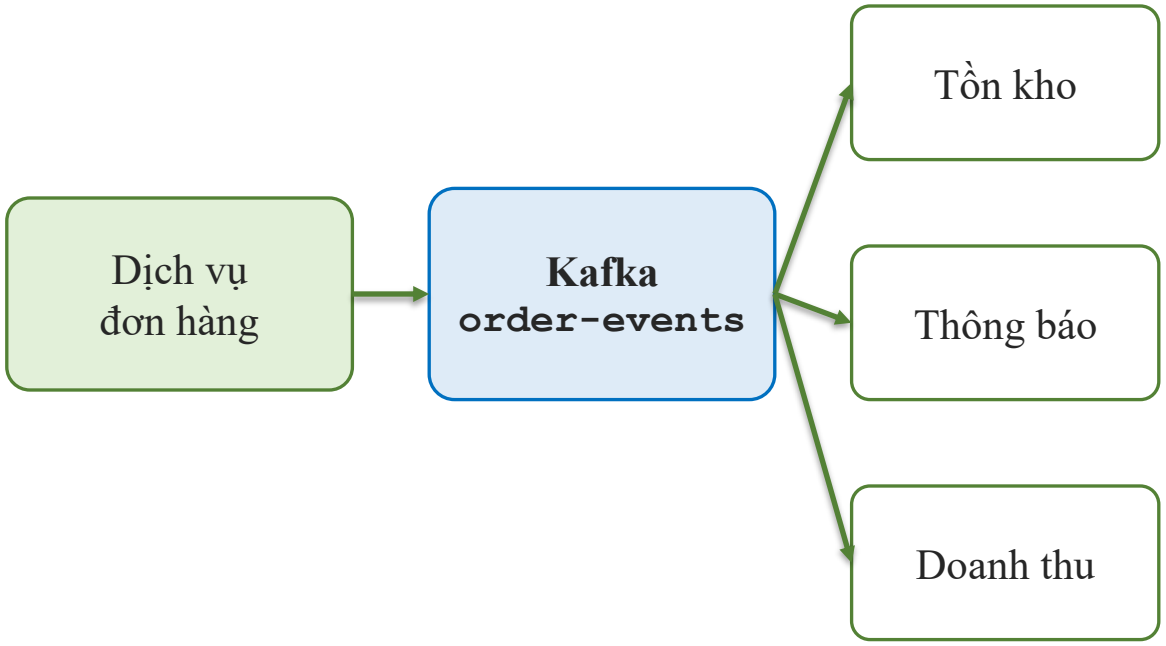
❖Đơn o1001 vừa được tạo. Ai cần biết?

Gọi trực tiếp



Thêm ứng dụng mới thì dịch vụ đơn hàng phải biết thêm một đích gọi. Một dịch vụ chậm thì phải chọn: chờ, thử lại hay bỏ qua.

Công bố sự kiện



Mỗi ứng dụng tự đọc sự kiện theo tốc độ của mình, kể cả đọc lại. Xử lý nghiệp vụ vẫn do mã ứng dụng thực hiện.

AI VIET NAM
@aivietnam.edu.vn

Kafka và kiến trúc

❖Kafka là gì? Một record gồm những gì?

Apache Kafka là nền tảng event streaming phân tán, làm ba việc:

1. nhận sự kiện do ứng dụng gửi đến;

2. lưu chúng theo cấu trúc có thứ tự;

3. cung cấp dữ liệu cho các ứng dụng đọc.

Thành phần	Ví dụ
Topic	`order-events`
Key	`o1001` – gom theo đơn
Value	JSON sự kiện bên phải
Timestamp	Thời gian của record

Value: một sự kiện OrderCreated

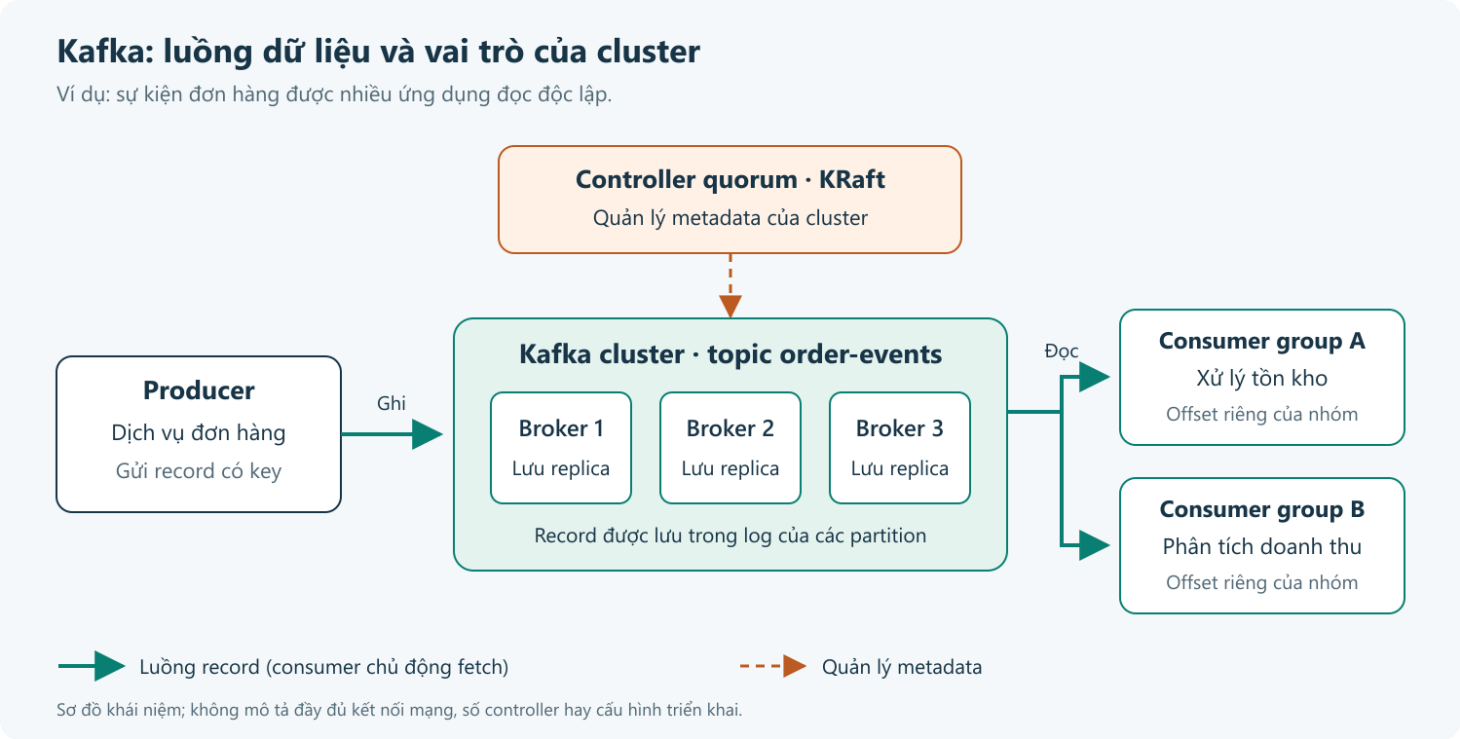
```
{
  "event_id": "e1001",
  "event_type": "OrderCreated",
  "order_id": "o1001",
  "total_amount": 450000,
  "currency": "VND",
  "occurred_at": "2026-11-02T09:00:00Z"
}
```

JSON chỉ là một cách biểu diễn value. `OrderCreated` và `OrderPaid` của cùng đơn có `event_id` khác nhau nhưng **cùng key**.

AI VIET NAM
@aivietnam.edu.vn

Kafka và kiến trúc

❖Producer, broker, controller và consumer



- **Broker**: tiến trình máy chủ lưu replica của partition, phục vụ đọc và ghi.
- **Cluster**: các broker phối hợp với nhau.
- **Controller (KRaft)**: quản lý metadata như thành viên cluster, leader của partition.
- Controller **không** là trạm trung chuyển mọi message.

Producer **gửi** · broker **lưu và phục vụ** · consumer **đọc và xử lý**.

Sơ đồ biên soạn cho bài học, dựa trên tài liệu thiết kế và KRaft của Apache Kafka 4.1.

Outline

SECTION 1

Kafka và kiến trúc

SECTION 2

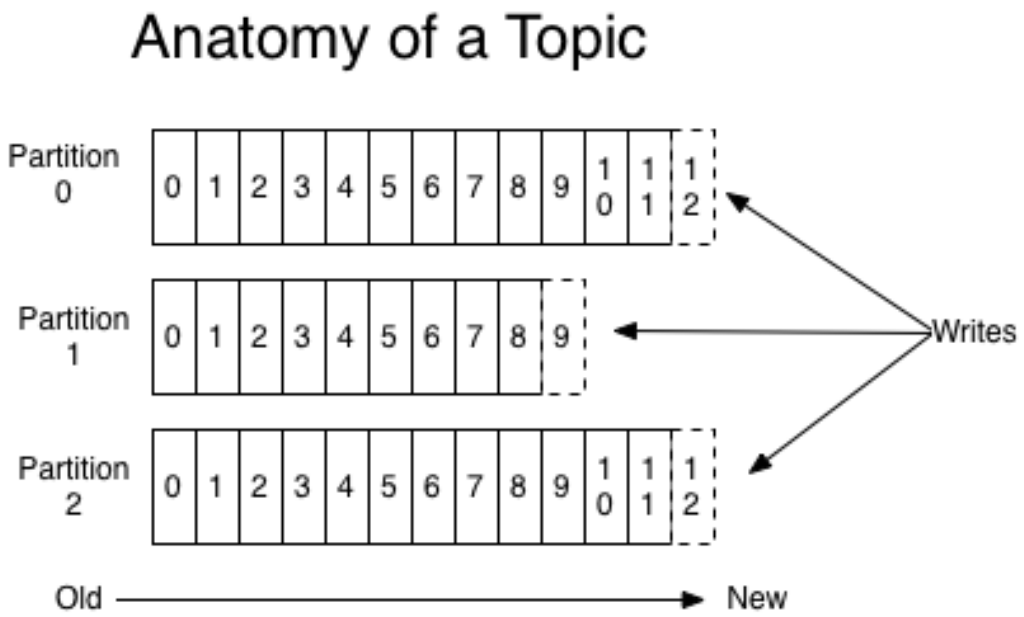
Topic, partition và key

SECTION 3

Replication và ISR

SECTION 4

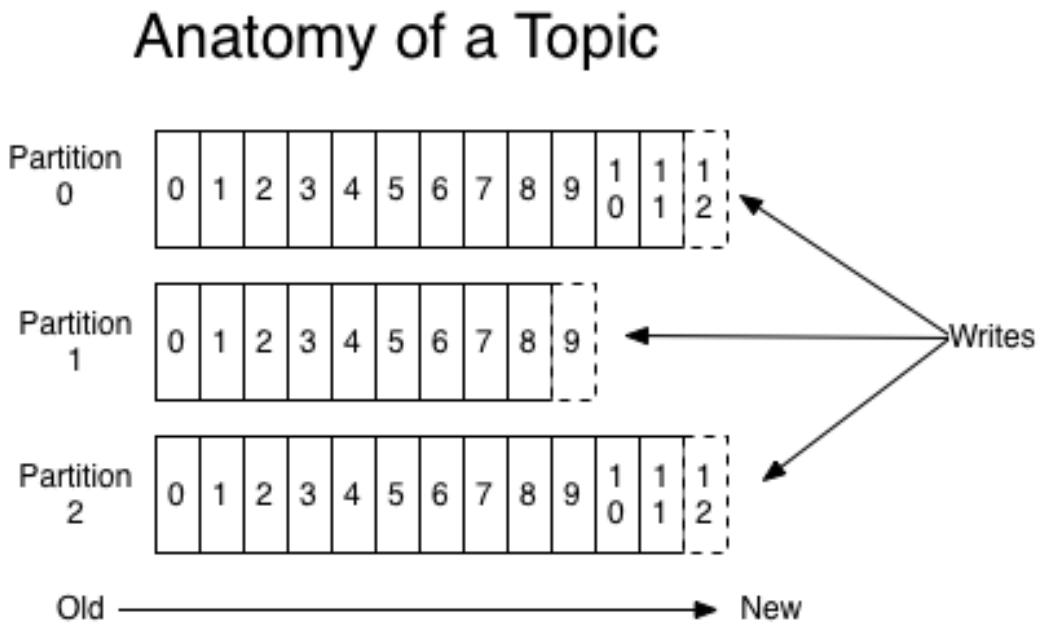
Offset và consumer group



AI VIET NAM
@aivietnam.edu.vn

Topic, partition và key

❖Topic được chia thành partition; mỗi partition là một log



Partition	Offset	Record
P0	0	Tạo đơn `o1001`
P0	1	Thanh toán `o1001`
P1	0	Tạo đơn `o1002`

- Record mới luôn được bổ sung vào **cuối** log.
- Địa chỉ của record = **(topic, partition, offset)**.
- `P0/offset 0` và `P1/offset 0` là hai vị trí khác nhau.

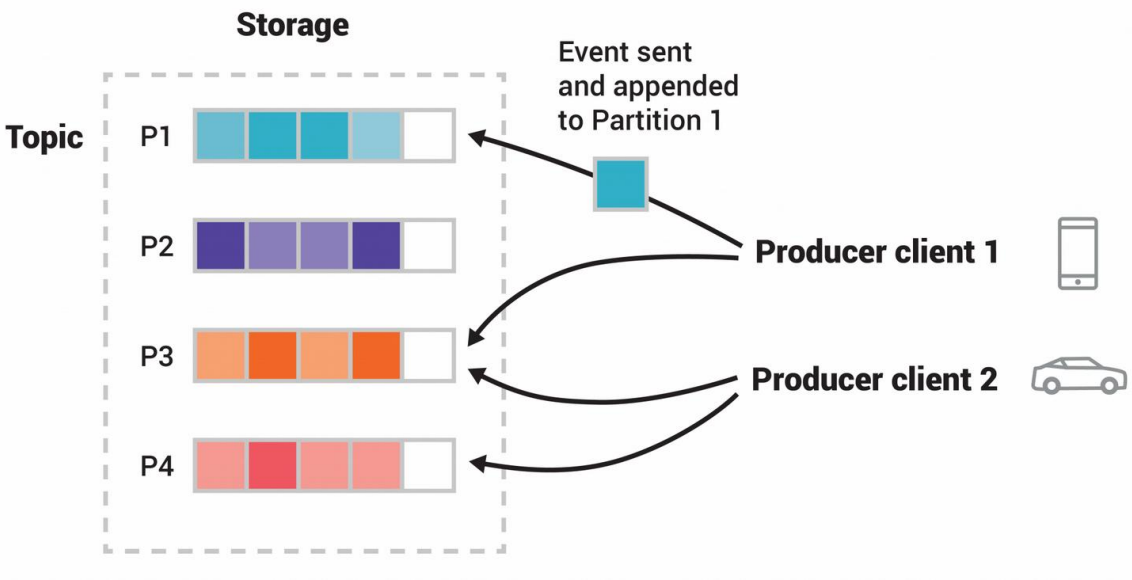
Nguồn: Apache Kafka, hình log_anatomy.png (hình gốc).

Kafka giữ thứ tự **trong từng partition**, không ghép thành một thứ tự toàn cục cho topic.

AI VIET NAM
@aivietnam.edu.vn

Topic, partition và key

❖Key quyết định record vào partition nào



Cùng key → cùng partition, **khi giữ nguyên** số partition, cách serialize key và partitioner.

Key	Điểm cần cân nhắc
`order_id`	Theo dõi vòng đời từng đơn
`customer_id`	Khách nhiều đơn có thể tạo tải lớn
Giá trị cố định	Dồn mọi record vào một partition

Nguồn: Apache Kafka, Introduction (hình gốc; Apache đặt tên P1–P4).

Không key ≠ round robin. Java producer mặc định chọn partition kiểu sticky, gắn với việc gom batch.

Tăng số partition có thể đổi ánh xạ key → partition; record cũ không tự được phân bố lại.

AI VIET NAM
@aivietnam.edu.vn

Topic, partition và key

❖Từ lab: 60 record, 30 key, 3 partition

Đếm record theo partition (lab, phần 5)

```
Name  Count
----  -----
0        20
1        22
2        18
```

Hai sự kiện của o0001

```
Partition:0|Offset:0|o0001|{"event_type":
  "OrderCreated",...}
Partition:0|Offset:1|o0001|{"event_type":
  "OrderPaid",...}
```

- 20/22/18, **không** phải 20/20/20: partitioner băm key, không chia lần lượt.
- Hai sự kiện của `o0001` cùng ở P0; `OrderCreated` có offset nhỏ hơn.
- Lượt đọc in P2 trước P0: thứ tự in giữa các partition không có ý nghĩa nghiệp vụ.

Lỗi gặp khi kiểm chứng: pipe từ PowerShell 5.1 gắn BOM vào key đầu tiên, biến `o0001` thành một key khác. Lab ghi file không BOM để tránh.

Outline

SECTION 1

Kafka và kiến trúc

SECTION 2

Topic, partition và key

SECTION 3

Replication và ISR

SECTION 4

Offset và consumer group



AI VIET NAM
@aivietnam.edu.vn

Replication và ISR

❖Partition khác replica



- **Replica**: một bản sao của partition.
- **Replication factor**: số replica của mỗi partition, tính cả leader.
- Mỗi partition có một **leader** nhận ghi; **follower** sao chép log từ leader.
- Một broker có thể là leader của P0 và giữ follower của P1, P2.

3 partition × RF 3 = **9 replica**, nhưng vẫn chỉ có **3 partition** logic.

3 partition, replication factor 3: 9 replica. Sơ đồ biên soạn theo mô hình replication của Kafka.

AI VIET NAM
@aivietnam.edu.vn

Replication và ISR

❖ISR: replica đang theo kịp leader

Lab mở rộng: 3 broker, trước khi dừng kafka-3

```
Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
Partition: 2  Leader: 3  Replicas: 3,1,2  Isr: 3,1,2
```

Sau docker stop kafka-3

```
Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2
Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,1
Partition: 2  Leader: 1  Replicas: 3,1,2  Isr: 1,2
```

- **ISR** (in-sync replicas): tập replica đang theo kịp leader theo tiêu chí của Kafka.
- `Replicas` không đổi: đó là danh sách **được gán**.
- P2 mất leader → broker 1 được bầu làm leader mới.

"Broker còn online" và "replica thuộc ISR" là hai thông tin khác nhau: replica còn chạy nhưng tụt lại vẫn có thể bị loại khỏi ISR.

AI VIET NAM
@aivietnam.edu.vn

Replication và ISR

❖acks và min.insync.replicas

Cấu hình	Producer chờ gì?
`acks=0`	Không chờ xác nhận
`acks=1`	Leader ghi vào log cục bộ
`acks=all`	Các replica trong ISR xác nhận

`min.insync.replicas`: ngưỡng ISR tối thiểu để chấp nhận ghi khi producer dùng `acks=all`. Không phải số replica được tạo ra.

Ví dụ: RF 3, `min.insync.replicas=2`, `acks=all`

ISR = 3 chờ cả ba replica trong ISR

ISR = 2 vẫn nhận ghi

ISR = 1 từ chối: `NOT_ENOUGH_REPLICAS`

`acks=all` liên quan đến ISR của partition, không phải mọi broker trong cluster. Producer nhận ack cũng chưa có nghĩa consumer đã xử lý xong.

AI VIET NAM
@aivietnam.edu.vn

Replication và ISR

❖Từ lab: ISR thấp hơn ngưỡng tối thiểu

kafka-3 dừng, ISR = 2, min.insync.replicas=3, gửi acks=all

```
WARN ... retrying (0 attempts left).
      Error: NOT_ENOUGH_REPLICAS
ERROR Error when sending message to topic
      lab-replicated ...
```

Gửi lại bằng acks=1, ngay sau đó xem offset cuối

```
lab-replicated:0:4
```

Sau docker start kafka-3, ISR trở lại 3

```
lab-replicated:0:5
```

- `acks=all` bị từ chối đúng như lý thuyết.
- `acks=1` được leader xác nhận, nhưng consumer **chưa đọc được** record.
- Kafka 4.1.2 bật **ELR** mặc định: high watermark chỉ tăng khi ISR đạt `min.insync.replicas`.

Ba mốc khác nhau: producer nhận ack, record đọc được, nghiệp vụ xử lý xong.

Outline

SECTION 1

Kafka và kiến trúc

SECTION 2

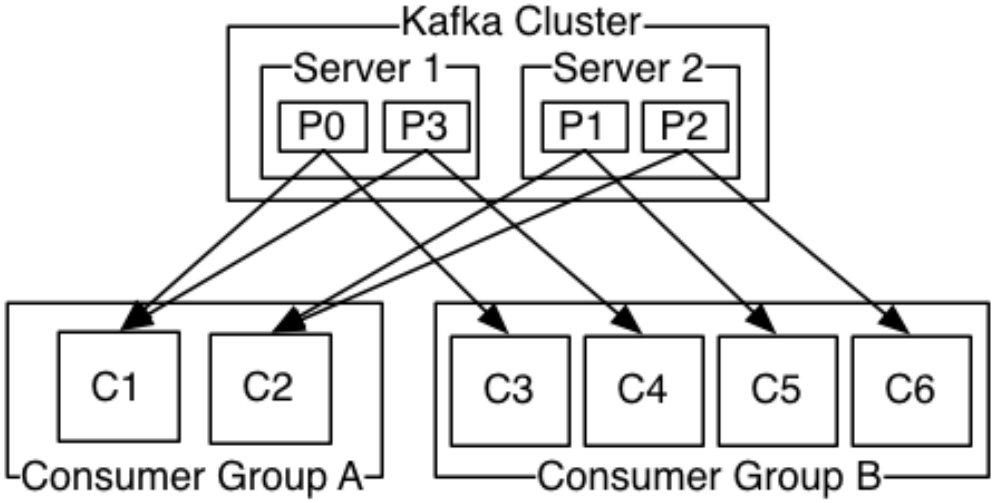
Topic, partition và key

SECTION 3

Replication và ISR

SECTION 4

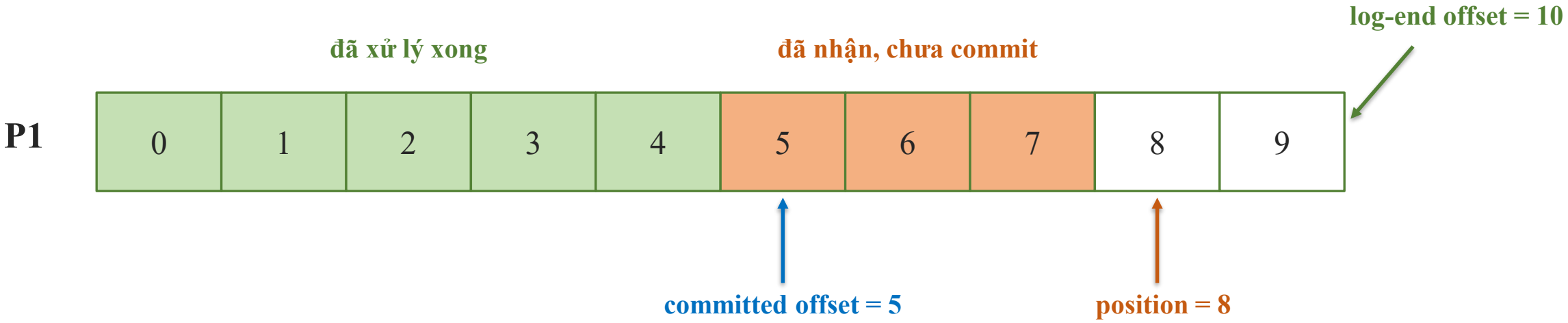
Offset và consumer group



AI VIET NAM
@aivietnam.edu.vn

Offset và consumer group

❖Consumer position và committed offset



- **Position**: offset của record tiếp theo consumer sẽ nhận.
- **Committed offset**: mốc đã lưu để group đọc tiếp khi khôi phục.
- Committed offset là **vị trí tiếp theo cần đọc**: xử lý xong 0–4 thì commit **5**.

Nếu consumer dừng lúc này, group đọc lại từ 5: record 5, 6, 7 có thể được **xử lý lần nữa**. Ứng dụng cần **idempotency**, ví dụ ghi nhận `event_id` đã xử lý.

AI VIET NAM
@aivietnam.edu.vn

Offset và consumer group

❖Consumer lag, retention và replay

Lab, phần 9: dừng consumer, gửi thêm 10 đơn

```
PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
0          21              22              1
1          22              25              3
2          19              25              6
```

Đặt lại committed offset rồi đọc lại

```
kafka-consumer-groups.sh --group lab-stock
  --topic lab-order-events --reset-offsets
  --to-earliest --dry-run   (rồi --execute)
```

Kết quả

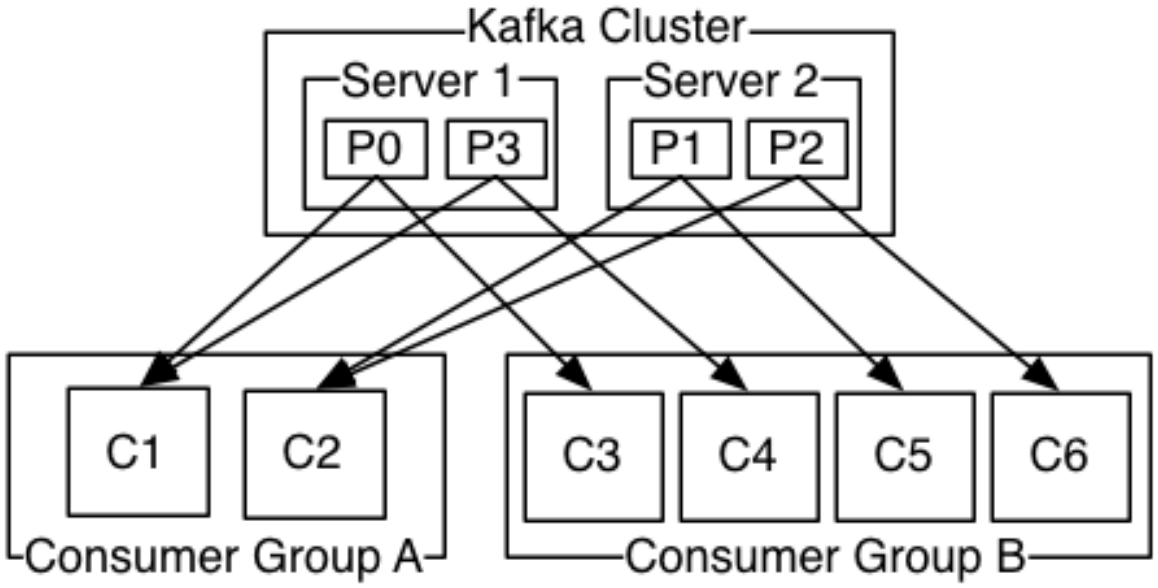
```
Processed a total of 72 messages   (60 + 2 + 10)
```

- **Lag** = log-end offset − committed offset, theo từng partition. Ở đây 1 + 3 + 6 = 10.
- Lag tính bằng offset, **không** bằng giây.
- Commit **không** xóa record; retention của topic quyết định việc dọn dữ liệu.
- **Replay** chỉ đọc lại được dữ liệu còn trong log.

AI VIET NAM
@aivietnam.edu.vn

Offset và consumer group

❖Một partition thuộc một consumer trong group



Số consumer	Phân công (3 partition)
1	C1: P0, P1, P2
2	C1: P0, P1; C2: P2
3	Mỗi consumer một partition
4	Một consumer không có partition

- Nhiều **group** đọc cùng topic độc lập, mỗi group có tiến độ riêng.
- **Rebalance**: phân công lại khi thành viên, subscription hoặc số partition thay đổi.

Nguồn: Apache Kafka, hình consumer-groups.png (hình gốc).

Thêm consumer vượt số partition không tăng thêm số bên đọc song song cho topic trong cùng group.

AI VIET NAM
@aivietnam.edu.vn

Offset và consumer group

❖Từ lab: hai, rồi bốn consumer trong lab-stock

Hai consumer

```
CLIENT-ID   #PARTITIONS   CURRENT-ASSIGNMENT
stock-c1    2             lab-order-events:0,1
stock-c2    1             lab-order-events:2
```

Bốn consumer, sau rebalance

```
CLIENT-ID   #PARTITIONS   CURRENT-ASSIGNMENT
stock-c1    1             lab-order-events:0
stock-c3    1             lab-order-events:2
stock-c4    0             -
stock-c2    1             lab-order-events:1
```

- `stock-c2` được giao P2 nhưng **không in record nào**: c1 đã đọc và commit hết 60 record trước khi c2 tham gia.
- Với bốn consumer, `stock-c4` có **0 partition**.
- `stock-c2` chuyển từ P2 sang P1 sau rebalance.
- Group `lab-analytics-01` vẫn đọc đủ 62 record.

AI VIET NAM
@aivietnam.edu.vn

Tổng kết

❖Theo dấu record o1001

Bước	Điều diễn ra	Khái niệm
1	Dịch vụ đơn hàng tạo record key `o1001`	Producer, key, value
2	Producer chọn partition đích, giả định P1	Topic, partitioner
3	Leader của P1 ghi record ở offset 7 (giả định)	Leader, log, offset
4	Follower sao chép; producer nhận ack theo cấu hình	Replica, ISR, `acks`
5	Consumer được giao P1 trong group tồn kho fetch và xử lý	Consumer group
6	Xử lý xong đến offset 7, ứng dụng commit **8**	Committed offset
7	Group phân tích đọc theo tiến độ riêng	Group độc lập, replay

Ba loại tiến độ: **Kafka xác nhận ghi**, **consumer đã nhận**, **nghiệp vụ đã xong**.

AI VIET NAM
@aivietnam.edu.vn

Tổng kết

❖Những điểm dễ nhầm

Dễ nhầm	Phù hợp
3 partition, RF 3 là 9 partition	3 partition logic, 9 replica
Cùng offset là cùng message	Offset chỉ có nghĩa trong partition của nó
Không key thì luôn round robin	Tùy client; Java mặc định dùng sticky
`acks=all`: mọi broker xác nhận	Liên quan đến ISR của partition
Commit offset 7 là xử lý xong record 7	Xử lý xong đến 7 thì commit 8
Commit xong thì record bị xóa	Retention quyết định việc dọn dữ liệu
Thêm consumer luôn nhanh hơn	Số partition giới hạn số bên đọc song song

AI VIET NAM
@aivietnam.edu.vn

Tổng kết

❖Kiểm tra nhanh

Q1 Topic có 6 partition, replication factor 2. Có bao nhiêu replica?

Q2 Group có 3 consumer đọc một topic 2 partition. Bao nhiêu consumer không có partition?

Q3 Ứng dụng xử lý xong offset 0–9 ở P0. Nên commit giá trị nào?

Q4 **LAG = 0** có chứng minh ứng dụng tồn kho đã cập nhật xong mọi đơn không?

Q5 Hai group đọc cùng topic. Group A commit offset 50 có ảnh hưởng gì đến group B?

AI VIET NAM
@aivietnam.edu.vn

Tổng kết

❖Đọc một sơ đồ Kafka bằng sáu câu hỏi

1. Ứng dụng nào phát sự kiện?

2. Dữ liệu vào topic nào?

3. Key phân phối record ra sao?

4. Partition được sao chép ở đâu?

5. Consumer chia nhóm thế nào?

6. Tiến độ được commit khi nào?

Tiếp theo: lab 90 phút

- Topic 3 partition, 60 record có key
- Hai và bốn consumer trong một group
- Lag, reset offset, replay
- Mở rộng: 3 broker, ISR, `acks`

Chuẩn bị: Docker Desktop đang chạy, image `apache/kafka:4.1.2`.

Tài liệu: bài đọc "Kiến trúc Apache Kafka", hướng dẫn hands-on, quiz 10 câu. Tham chiếu: kafka.apache.org/41